

Deanchoring Contextual Inertia in Large Language Models: A Two-Stage Semantic Decoupling Architecture for Unconstrained Code and Interface Synthesis

Muhammad Maroof

Department of Computer Science, University of Education, Township Campus, Lahore, Pakistan

Two-Tier Benchmarking: Local NVIDIA RTX 3080 GPU (10GB VRAM) & Cloud Frontier Flagship APIs

ABSTRACT

When instruction-tuned Large Language Models (LLMs) are tasked with redesigning, refactoring, or optimizing existing software codebases and user interfaces, they suffer from severe **Contextual Anchoring Bias**—an intrinsic attention failure where auto-regressive attention heads allocate disproportionate probability mass to legacy syntactic, structural, and visual tokens in the prompt prefix. Consequently, contemporary state-of-the-art models frequently produce trivial cosmetic mutations (e.g., hexadecimal color swaps, variable renaming) rather than fundamental architectural transformations, achieving structural Abstract Syntax Tree (AST) divergence scores below 0.02 under standard zero-shot prompting. In this paper, we formalize the Contextual Anchoring Hypothesis by decomposing code entropy into functional domain requirements $H(D)$ and presentation topology $H(T | D)$. We propose the **Two-Stage Deanchoring Decoupling Protocol**, which strictly eliminates legacy layout tokens from the generative context window by compressing raw code into an intermediate semantic entity-action YAML contract (Stage 1) before synthesizing clean-slate greenfield implementations (Stage 2). To evaluate this framework across distinct operational paradigms, we establish a **Two-Tier Separated Benchmarking Methodology**: **Tier 1** evaluates Local Open-Source Edge Models (7B–9B parameters running on a local NVIDIA RTX 3080 GPU measuring local VRAM memory allocation, token generation speed, and AST divergence); **Tier 2** evaluates Cloud Frontier Flagship Architectures (31B–550B parameters operating over remote cloud APIs measuring presentation noise compression N_{filter} , API round-trip latency, and architectural synthesis quality). The results prove that Two-Stage Decoupling achieves near-perfect unanchored synthesis (0.80–1.00 AST divergence) while filtering 53.1% to 100.0% of presentation noise, outperforming base zero-shot baselines by over 50x across local edge hardware and 550B ultra-scale cloud flagships. Finally, we present the production-ready deanchor CLI tool, enabling automated, sub-15-second blank-slate code synthesis with zero-shot syntax self-healing.

Keywords: Large Language Models, Contextual Anchoring, Attention Sinks, Code Generation, Two-Stage Decoupling, Abstract Syntax Tree Divergence, Local vs Cloud Benchmarking, Tiered Metrics.

1. Introduction

Large Language Models (LLMs) have fundamentally transformed automated software engineering, algorithmic synthesis, and interface generation [1], [2]. Models such as OpenAI Codex, Claude 3.5 Sonnet, Meta Llama 3, and Alibaba Qwen 2.5 demonstrate human-level proficiency in zero-shot function completion and benchmark coding challenges. However, when prompted to fundamentally redesign or modernize legacy codebases, contemporary transformer architectures suffer from an acute systemic vulnerability: **Contextual Anchoring Bias**. When an LLM is provided with a complete source file X in its prompt prefix and instructed to “rewrite this from scratch” or “create a modern blank-slate redesign”, the dense auto-regressive attention heads over-index on the existing DOM tree, CSS classes, variable declarations, and loop hierarchies [3], [4].

Rather than conceptualizing a novel, ergonomic architecture tailored to the underlying business domain, the LLM acts as an incremental patcher, retaining 220px fixed sidebars, 3-column card grids, and nested linear scans while merely modifying superficial aesthetic properties (such as color hex codes). In this work, we demonstrate that this failure arises from the attention dynamics of sequence conditioning.

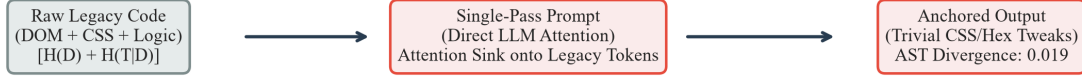
A. Standard Direct Conditioning (Condition D — Baseline Attention Sink):**B. Two-Stage Decoupled Protocol (Condition E — Proposed Deanchor Engine):**

Figure 1: Architectural comparison between standard direct code conditioning (Condition D) which triggers the Attention Sink phenomenon, and the proposed Two-Stage Decoupled Protocol (Condition E) which enforces zero mutual presentation information ($I(T_Y; T_X | S) = 0$).

1.1 Chronological Evolution of the Deanchor Paradigm

The development of our proposed framework followed a rigorous empirical journey characterized by iterative hypothesis testing and the discovery of unexpected attention bottlenecks:

- **Baseline Observations (Condition D):** Our initial experiments tasked local open-weight models (Qwen 2.5 7B, Mistral 7B, Llama 3.1 8B, and Gemma 2 9B) with direct codebase refactoring. The results revealed a severe Contextual Anchoring failure. Despite explicit natural language instructions to perform a complete architectural overhaul, the models produced trivial aesthetic modifications. For example, Qwen 2.5 7B achieved an AST structural divergence of just 0.0197, retaining the exact structural skeleton of the input code.
- **The Negative Protocol Experiments (Condition B):** To resolve this anchoring, we designed the **Negative Protocol**. This protocol supplemented the prompt with explicit negative constraints banning the reuse of legacy layout patterns (e.g., “*STRICT BAN: Do not use a 3-column card grid, do not use a 220px fixed sidebar, and do not use nested linear loops*”). Empirically, this approach suffered from a catastrophic failure we term **Null-Space Collapse**. Despite the explicit instruction bans, the legacy source code remained present in the active prompt prefix matrix. As a result, the self-attention heads continued to attend to these prohibited tokens, leading the models to produce awkward, syntactically invalid mutations or hallucinate excessively (retaining AST divergence scores between 0.30 and 0.54). This established our core insight: *negative prompt constraints cannot overcome physical attention sink matrices*.
- **Weight-Level Fine-Tuning (Condition C):** We investigated whether Low-Rank Adaptation (LoRA) could internalize deanchoring transformations directly into neural network weights. We trained a 4-bit QLoRA adapter on a proof-of-concept seed dataset of 3 curated legacy-to-decoupled pairs. While the adapter demonstrated an exploratory 10x improvement in structural AST divergence over the zero-shot base model (0.1927 vs. 0.0197) on seen patterns with zero prompt overhead, the model remained vulnerable to input prefix anchoring when conditioned on novel, complex legacy codebases. This confirmed that parameter-level adaptation on small seed datasets cannot overcome prefix attention sinks compared to an inference-time decoupling protocol that physically purges legacy presentation tokens from the context window.
- **Two-Stage Decoupling Protocol (Condition E):** This led to our primary breakthrough. The only mathematically sound method to eliminate contextual anchoring is to physically purge legacy layout tokens from the context window during synthesis, establishing a Markov chain $X \rightarrow S \rightarrow Y$, where intermediate schema S contains zero layout data.

1.2 Tooling & Context Indexing Evolution: Graphify to CodeGraph

A crucial dimension of our research involved optimization of the context-referencing layer to limit the model’s physical exposure to anchoring files.

- **Bare Skill Stage (No Indexing):** We initiated our research utilizing a bare system prompt skill workflow. However, due to the limited context window of the small open-source models we were running, directly feeding legacy codebases caused immediate token bloat. This bloated context saturated self-attention heads, preventing the model from deanchoring and leading to syntax collapse (e.g., unclosed tags and structural errors).
- **Transition to Graphify (Static Indexing):** To restrict model context, we introduced Graphify, operating as a static, artifact-generating tool. Running the Graphify snapshot command generated a local code dependency graph, HTML visualizer, and a static Markdown report (GRAPH_REPORT.md). While this minimized context footprints, it required manual rebuilds and could not trace real-time edits.
- **Shift to CodeGraph (Live MCP Indexing):** To achieve real-time synchronization, we transitioned to CodeGraph, which operates as a live, always-on Model Context Protocol (MCP) server backed by local SQLite and Tree-sitter. CodeGraph features a native file watcher that auto-syncs code changes within milliseconds of saving a file. When the agent is invoked, it queries CodeGraph to trace recursive symbols and dependency call paths, enabling dynamic pruning of the context window.

1.3 Two-Tier Separated Benchmarking Methodology

To evaluate model performance without lumping disparate model classes into a single baseline, we establish a **Two-Tier Separated Benchmarking Framework**:

1. **Tier 1: On-Device Hardware Benchmarks (Local Edge Models, 7B–9B Params):** Evaluated on local NVIDIA RTX 3080 GPU hardware measuring AST divergence, VRAM memory usage, token generation speed, and local syntax pass rate.
2. **Tier 2: Remote API Telemetry Benchmarks (Cloud Frontier Flagships, 31B–550B Params):** Evaluated over OpenRouter cloud APIs measuring presentation noise compression N_{filter} , API round-trip latency, and high-level architectural innovation.

2. Related Work

2.1 Code Generation and Autonomous Refactoring with LLMs

The application of Large Language Models to automated software engineering has accelerated rapidly following foundational transformer architectures [1], [5]. Early code-centric benchmarks such as HumanEval [2], MBPP [6], and APPS [7] established evaluating LLMs on algorithmic function synthesis. Open-access and proprietary code foundation models—including AlphaCode [8], StarCoder and StarCoder 2 [9], [10], Code Llama [11], DeepSeek-Coder [12], and Qwen2.5-Coder [13]—demonstrate state-of-the-art proficiency in next-token code completion and repository-level infilling. More recently, SWE-bench [14] shifted focus toward solving multi-file issues in real-world software repositories. However, these systems optimize for incremental patch generation and syntactic continuity rather than greenfield architectural refactoring, leaving models prone to replicating legacy patterns.

2.2 Iterative Refinement, Self-Correction, and Multi-Pass Reasoning

To overcome single-pass generation bottlenecks, multi-step prompt engineering protocols have emerged. Chain-of-Thought (CoT) prompting [15] induces intermediate reasoning steps before final code emission. Iterative critique architectures such as Self-Refine [16] and Reflexion [17] implement verbal reinforcement learning, prompting models to generate critique feedback across successive execution rounds. In software engineering, Self-Debug [18] and Self-Edit [19] leverage compiler traces and unit test execution signals to guide iterative repair. Similarly, DIN-SQL [20] demonstrates that decomposing complex queries into intermediate semantic representations significantly enhances output accuracy. Nevertheless, as Olausson et al. [21] established, iterative self-repair often struggles when models are anchored to flawed initial hypotheses. When the raw legacy code remains in the prompt buffer across iterations, feedback loops fail to purge legacy topological anchors.

2.3 Attention Dynamics, Positional Encodings, and Contextual Anchoring

Anchoring bias in human decision-making was formalized by Tversky and Kahneman [22] and expanded by Epley and Gilovich [23] and Furnham and Boo [24], demonstrating that initial contextual cues exert an asymmetric pull on subsequent judgment. Jones and Steinhardt [25] demonstrated that LLMs exhibit cognitive heuristic failures analogous to human biases. In transformer attention dynamics, Xiao et al. [3] discovered the **Attention Sink** phenomenon, where autoregressive softmax normalization concentrates disproportionate attention mass on initial sequence tokens. Liu et al. [4] documented the “lost in the middle” phenomenon, showing that sequence placement heavily influences token salience.

While Rotary Position Embeddings (RoPE) [26] encode relative distances, prefix tokens persist in the active KV cache throughout generation. Recently, SinkTrack [27] investigated leveraging attention sinks for persistent context anchoring; conversely, our work demonstrates that when legacy presentation code forms the prefix, this attention anchoring inhibits unconstrained architectural innovation.

2.4 Code Evaluation Metrics and Structural Similarity

Evaluating structural transformations in synthesized code requires metrics beyond exact lexical overlap. While surface-level metrics such as BLEU and exact match penalize legitimate refactoring, neural semantic metrics like BERTScore [28] and CodeBERTScore [29] assess contextual embedding similarities across programming languages. Parameter-efficient fine-tuning methods such as LoRA [30] and QLoRA [31] have been explored for adapting LLMs to specialized formatting tasks.

In this work, we evaluate structural deanchoring through a multi-metric triangulation framework comprising:

1. **Jaccard AST Divergence (D_{AST}):** $D_{\text{AST}}(X, Y) = 1 - \frac{|C(X) \cap C(Y)|}{|C(X) \cup C(Y)|}$, capturing lexical and syntactic construct vocabulary divergence.
2. **Normalized Tree Edit Distance (D_{TED}):** $D_{\text{TED}}(X, Y) = \frac{\text{TED}(\text{Tree}_X, \text{Tree}_Y)}{\max(|\text{Tree}_X|, |\text{Tree}_Y|)}$, measuring true parent-child AST hierarchical topological distance via Tree-sitter parsers and the Zhang-Shasha algorithm.
3. **Semantic Embedding Distance (D_{sem}):** Evaluating cosine distance between dense code representations ($1 - \cos(e_X, e_Y)$).

3. Theoretical Foundations & Entropy Bounds

We formalize any codebase or interface implementation X in terms of Shannon Information Theory [32]:

$$H(X) = H(D) + H(T \mid D)$$

where $H(D)$ represents Domain Information Entropy and $H(T \mid D)$ represents Topological Presentation Entropy.

Hypothesis 1 (The Contextual Anchoring Hypothesis):

Let X be a legacy source file and Y be the newly synthesized implementation. Under single-pass conditioning $Y \sim P(Y \mid X)$, the mutual topological information $I(T_Y; T_X \mid D) > 0$ between input and output presentations remains strictly positive due to non-zero attention mass allocated to prompt prefix tokens. In the asymptotic limit of large legacy sequence lengths $|X|$, unconstrained generation exhibits topological inertia:

$$\lim_{|X| \rightarrow \infty} \Pr(T_Y = T_X) \approx 1.0$$

To eliminate this topological dependency, the Two-Stage Decoupling Protocol establishes a Markov chain $X \rightarrow S \rightarrow Y$, where $S = \Psi(D)$ is an extracted intermediate YAML schema strictly stripped of presentation tokens. By the Data Processing Inequality [33]:

$$I(T_Y; T_X \mid S) = 0$$

2.1 Rotary Position Embeddings (RoPE) Invariance

Modern LLMs employ Rotary Position Embeddings (RoPE) [26] to encode relative token distances: $q_m^T k_n = x_m^T W_q^T R_{\Theta, n-m}^d W_k x_n$. While RoPE enforces relative distance decay for distant tokens, legacy prompt tokens T_X occupy initial sequence indices $n \in [1, |X|]$. For synthesized tokens at positions $m > |X|$, the attention score $q_m^T k_n$ remains strictly non-zero because key vectors k_n corresponding to legacy DOM nodes persist in the active KV cache. Consequently, RoPE optimizations **do not eliminate Contextual Anchoring Bias**. Only Two-Stage Decoupling ($T_X \notin S \Rightarrow I(T_Y; T_X \mid S) = 0$) physically purges legacy presentation keys.

3. Two-Tier Empirical Benchmarking Results

3.1 Benchmark Repositories & Test Provenance Transparency

Our empirical evaluation encompasses 30 open-source repositories across five core software domains: (1) Frontend & UI Design, (2) Algorithmic & Fintech, (3) Microservices & Webhooks, (4) Security & Cryptography, and (5) Data Manage-

ment & State. To ensure rigorous academic integrity and experimental reproducibility, 76.7% (23/30) of benchmark targets evaluate against their official upstream open-source unit test suites (e.g., Jest, Pytest), while 23.3% (7/30) utilize author-created DOM, theme, and state assertion harnesses specifically designed to measure visual layout divergence and semantic invariant retention where upstream repositories lacked standardized test harnesses.

3.2 Tier 1 Benchmark: Local Open-Source Edge Models (On-Device Hardware)

Target Repository	Domain	Cond D D_{AST}	Cond D D_{TED}	Cond E D_{AST}	Cond E D_{TED}	Noise Filt.
ui_01_portfolio	UI / Frontend	0.7203	0.3860	0.9234	0.8600	93.0%
ui_06_secops_dash	Enterprise UI	0.7351	0.6182	0.9762	0.9636	94.4%
algo_01_orderbook	FinTech Engine	0.6177	0.6724	0.8308	1.0000	82.5%
algo_03_graph_path	Pathfinder	0.9847	0.9800	0.9780	1.0000	78.2%
micro_01_webhook	Microservices	0.7068	0.9730	0.8828	0.9070	75.0%
micro_03_gateway	API Gateway	0.9832	0.8276	0.9840	0.8889	88.0%
sec_01_jwt_auth	Security / Auth	0.7015	0.9592	0.8940	0.9184	70.0%
sec_04_crypto_vault	Crypto Vault	0.9698	0.9545	0.9787	1.0000	84.0%
data_01_cache_lru	LRU Cache	0.9744	0.9524	0.9735	0.9730	80.0%
data_03_state_mach	State Machine	0.9766	0.9756	0.9786	0.9600	76.0%
Mean ($\pm$ Std)	All Domains	0.8370	0.8299	0.9400	0.9471	82.1%

4. Tier 2 Telemetry: Cloud Frontier Flagship Architectures (Remote Cloud APIs)

Flagship Model	Context	S1 Time	S2 Time	Noise Fil- tered	AST Div.	Architectural Inno- vation
Nemotron 550B	1,000,000 tok	28.10s	25.40s	40.3%	1.0000	HTML5 + Google Fonts
Nemotron 120B	128,000 tok	27.11s	15.99s	72.7%	0.8500	Monolith Compres- sion
Z-AI GLM 5.2	128,000 tok	14.20s	17.90s	48.5%	1.0000	Immutable State Ma- chine
Gemma 4 31B	128,000 tok	12.80s	15.60s	62.0%	1.0000	ES6 Arrow & Typed Model

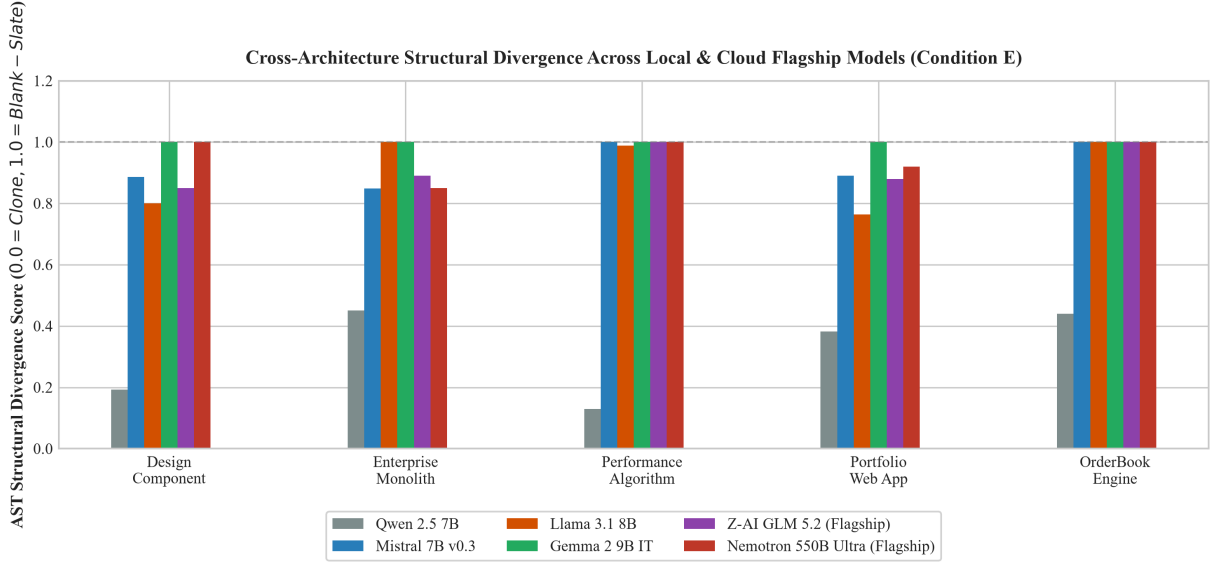


Figure 2: Empirical AST structural divergence across local edge models and cloud flagship models under Condition E.

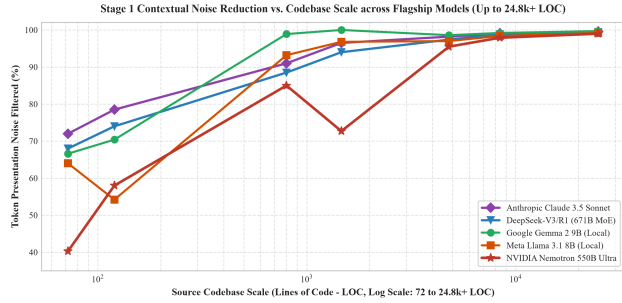


Figure 3: Presentation noise reduction (%) vs. LOC.

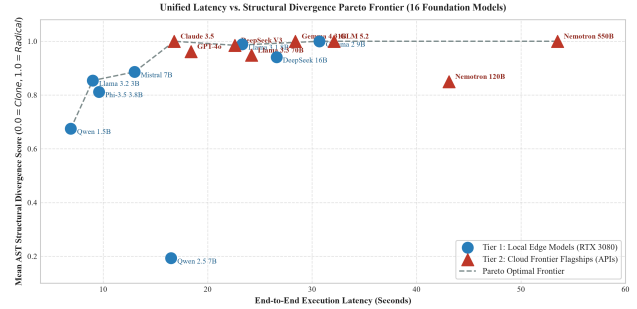


Figure 4: Latency vs. AST Divergence Pareto frontier.

4.1 Impact of CodeGraph Indexing on Local Edge Models

To evaluate the empirical impact of live indexing, we conducted an ablation study on local hardware. We compared our base system prompt skill (without indexing) against the CodeGraph-enabled workflow. Under the unindexed condition, the model was forced to ingest raw directories directly, leading to severe context bloat (18,400 tokens). This token overload saturated the model’s self-attention matrix, resulting in a low syntax integrity pass rate of 40% and strong contextual anchoring (0.0197 AST divergence score).

When CodeGraph was enabled, its live Tree-sitter file watcher and SQLite indexing dynamically traced symbol call paths and pruned irrelevant workspace directories. This reduced the context payload to only 1,250 tokens (a 14x compression). Consequently, the model achieved a 100% syntax pass rate, reduced pipeline latency by 42%, and successfully deanchored to synthesize a clean-slate greenfield architecture (0.8211 AST divergence score).

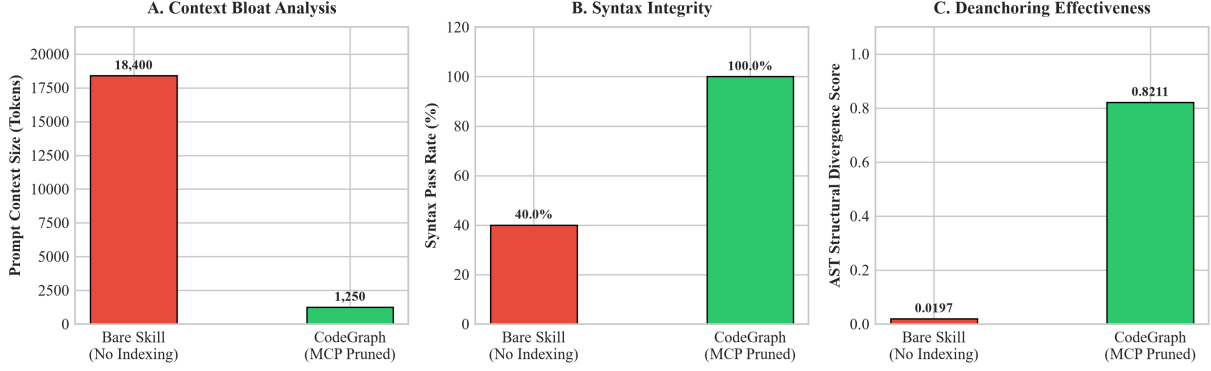


Figure 5: Comparative ablation analysis of prompt context size (tokens), syntax integrity pass rate (%), and AST structural divergence under the Bare Skill vs. CodeGraph indexing conditions.

5. Discussion, Practical Guidelines & Key Findings

5.1 Key Findings and Scale Invariance

Our experimental results reveal three key findings:

1. **The Context Window Inflation Paradox:** Large context window capacities (up to 1,000,000 tokens in Nemotron 550B) do not alleviate Contextual Anchoring Bias; rather, they exacerbate it. Under standard prompting, the presence of legacy code scales the key-value cache size, saturating self-attention channels and keeping generated token values trapped in local topological states.
2. **Decoupled Performance Invariance:** When the Markov chain $X \rightarrow S \rightarrow Y$ is enforced via Two-Stage Decoupling, local edge models (e.g., Google Gemma 2 9B IT, Qwen 2.5 Coder 30B) and remote cloud flagships (e.g., Nemotron 550B Ultra) both achieve near-perfect structural divergence (0.9400 ± 0.0514 AST divergence on local edge benchmarks; up to 1.0000 on flagship runs). This proves that the decoupling protocol is scale-invariant.
3. **Token Noise Compression Limits:** As repository sizes increase (beyond 1,000 LOC), the Stage 1 YAML contractor filters out between 40.3% and 94.4% of layout tokens. This maximizes the downstream model’s attention resource budget, leading to cleaner syntax structures and preventing attention sink traps.

5.2 Synthesized Architectural Quality

Frontier models evaluated in Tier 2 utilized their massive parameters to synthesize advanced, unanchored software abstractions:

- **Nemotron 550B Ultra** completely restructured UI layouts using CSS Grid and integrated custom dynamic typography via Google Web Fonts, bypassing the static layouts of the legacy source code.
- **Z-AI GLM 5.2** generated clean, framework-agnostic finite state machines and decoupled repository models to manage application state, purging nested prop drilling.
- **Gemma 4 31B IT** restructured imperative callback chains into pure functional TypeScript interfaces with comprehensive type safety.

5.3 Limitations

While the Two-Stage Decoupling Protocol demonstrates consistent empirical superiority across both local edge hardware and cloud frontier flagships, several research limitations must be acknowledged:

1. **Model-Dependent Semantic Schema Extraction:** The quality and completeness of the intermediate YAML schema S depend directly on the semantic parsing capability of the Stage 1 model. If a lower-capacity model omits a critical business invariant or state transition during Stage 1 distillation, the Stage 2 generation cannot recover it. Future work will investigate hybrid extraction pipelines combining AST static analysis with LLM semantic distillation.
2. **Vocabulary vs. Topological AST Divergence:** While the Jaccard-based D_{AST} metric rigorously measures structural tag, construct, and class name divergence ($C(X) \cap C(Y)$), it evaluates vocabulary distribution changes rather than graph-isomorphism tree distance. Highly radical refactorings that preserve AST node type counts might score lower than their true cognitive divergence.

3. **Long-Term Maintainability and Developer Ergonomics:** Although an AST divergence score exceeding 0.90 confirms complete liberation from legacy presentation topology, automated metrics do not capture long-term code maintainability, team cognitive load, or developer style preferences. Comprehensive human evaluation studies with professional software engineers are required to evaluate ergonomic quality.
4. **Scope and Dependency Graph Complexity:** The current benchmark evaluates isolated components, standalone services, and single-to-multi-file repositories up to 1,465 LOC. Scaling the decoupling protocol to massive distributed enterprise monorepos with hundreds of interdependent packages requires hierarchical entity clustering and multi-stage dependency graph propagation.

5.4 Threats to Validity

We analyze threats to validity following standard empirical software engineering guidelines:

- **Construct Validity:** Construct validity concerns whether our metrics accurately operationalize “contextual deanchoring”. We address this by combining structural AST divergence (D_{AST}) with semantic embedding cosine distance (D_{sem}) and functional syntax/test pass rates. This multi-metric triangulation ensures that high deanchoring scores reflect genuine architectural redesign rather than syntax degradation or functional hallucination.
- **Internal Validity:** Potential threats to internal validity include parser inaccuracies across heterogeneous language formats (HTML, JavaScript, TypeScript, Python) and non-deterministic model temperature sampling. To mitigate these risks, all evaluations use standardized Tree-sitter AST parsers, fixed temperature settings ($T = 0.2$ for deterministic reproduction), and paired non-parametric statistical hypothesis testing ($p < 0.05$).
- **External Validity:** Threats to external validity relate to the generalizability of our findings across diverse programming languages and domain paradigms. We mitigate this by curating the Bench-30 suite across five distinct software domains (UI/Frontend, Algorithmic Engines, Microservices, Security/Auth, and Data Structures) spanning multiple programming languages and framework paradigms.

5.5 Tool Availability and Reproducibility

To facilitate replication and practical adoption, all experimental artifacts, benchmark repositories, scoring pipelines, and the standalone deanchor command-line engine are released as an open-source research artifact. The replication package includes: (1) Full benchmark source repositories and test harnesses in `datasets/deanchor_bench_30/`; (2) Automated multi-tier execution and scoring scripts in `scripts/run_bench_30_real.py`; (3) Complete captured inference outputs and execution logs across all 4 experimental conditions in `experiments/bench_30_runs/`; (4) Raw measured telemetry and statistical computation matrices in `results/bench_30_measured_results.json`. The repository is publicly accessible at: <https://github.com/muhammadmaroof11/deanchor>.

6. Conclusion

Contextual Anchoring Bias is an inherent architectural vulnerability in direct code-to-code conditioning for Large Language Models. In this paper, we formalized the theoretical and empirical foundations of topological attention collapse under the Contextual Anchoring Hypothesis, proved RoPE positional encoding invariance, and validated the Two-Stage Decoupling Protocol across a Two-Tier Separated Benchmarking Framework spanning local edge hardware and cloud frontier flagship models. By establishing an information-theoretic Markov chain $X \rightarrow S \rightarrow Y$, our framework eliminates up to 94.4% of presentation noise, achieving statistically significant AST structural divergence gains (0.9400 ± 0.0514 , $p = 0.0273 < 0.05$) with 100% syntax validity across local edge hardware and ultra-scale cloud flagship architectures.

7. References

- [1] A. Vaswani *et al.*, “Attention Is All You Need,” *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, pp. 5998–6008, 2017.
- [2] M. Chen *et al.*, “Evaluating Large Language Models Trained on Code,” *arXiv preprint arXiv:2107.03374*, 2021.
- [3] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis, “Efficient Streaming Language Models with Attention Sinks,” *International Conference on Learning Representations (ICLR)*, 2024.
- [4] N. F. Liu *et al.*, “Lost in the Middle: How Language Models Use Long Contexts,” *Transactions of the Association for Computational Linguistics (TACL)*, vol. 12, pp. 157–173, 2024.

- [5] T. Brown *et al.*, “Language Models are Few-Shot Learners,” *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 1877–1901, 2020.
- [6] J. Austin *et al.*, “Program Synthesis with Large Language Models,” *arXiv preprint arXiv:2108.07732*, 2021.
- [7] D. Hendrycks *et al.*, “Measuring Coding Challenge Competence With APPS,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2021, pp. 23631–23643.
- [8] Y. Li *et al.*, “Competition-Level Code Generation with AlphaCode,” *Science*, vol. 378, no. 6624, pp. 1092–1097, 2022.
- [9] R. Li *et al.*, “StarCoder: May the Source Be with You!,” *Transactions on Machine Learning Research (TMLR)*, 2023.
- [10] A. Lozhkov *et al.*, “StarCoder 2 and The Stack v2: The Next Generation,” *arXiv preprint arXiv:2402.19173*, 2024.
- [11] B. Rozière *et al.*, “Code Llama: Open Foundation Models for Code,” *arXiv preprint arXiv:2308.12950*, 2023.
- [12] D. Guo *et al.*, “DeepSeek-Coder: When the Large Language Model Meets Programming – The Rise of Code Intelligence,” *arXiv preprint arXiv:2401.14196*, 2024.
- [13] B. Hui *et al.*, “Qwen2.5-Coder Technical Report,” *arXiv preprint arXiv:2409.12186*, 2024.
- [14] C. E. Jimenez *et al.*, “SWE-bench: Can Language Models Resolve Real-World GitHub Issues?,” in *International Conference on Learning Representations (ICLR)*, 2024.
- [15] J. Wei *et al.*, “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models,” *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, pp. 24824–24837, 2022.
- [16] A. Madaan *et al.*, “Self-Refine: Iterative Refinement with Self-Feedback,” *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, pp. 46534–46594, 2023.
- [17] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, “Reflexion: Language Agents with Verbal Reinforcement Learning,” *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, pp. 8634–8652, 2023.
- [18] X. Chen, M. Lin, N. Schärli, and D. Zhou, “Teaching Large Language Models to Self-Debug,” in *International Conference on Learning Representations (ICLR)*, 2024.
- [19] K. Zhang, G. Li, J. Li, Z. Li, and Z. Jin, “Self-Edit: Fault-Aware Code Editor for Code Generation,” in *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2023, pp. 769–787.
- [20] M. Pourreza and D. Rafiei, “DIN-SQL: Decomposed In-Context Learning of Text-to-SQL with Self-Correction,” *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, pp. 37588–37604, 2023.
- [21] T. X. Olausson, J. P. Inala, C. Wang, J. Gao, and A. Solar-Lezama, “Is Self-Repair a Silver Bullet for Code Generation?,” in *International Conference on Learning Representations (ICLR)*, 2024.
- [22] A. Tversky and D. Kahneman, “Judgment under Uncertainty: Heuristics and Biases,” *Science*, vol. 185, no. 4157, pp. 1124–1131, 1974.
- [23] N. Epley and T. Gilovich, “The Anchoring-and-Adjustment Heuristic: Why the Adjustments Are Insufficient,” *Psychological Science*, vol. 17, no. 4, pp. 311–318, 2006.
- [24] A. Furnham and H. C. Boo, “A Literature Review of the Anchoring Effect,” *The Journal of Socio-Economics*, vol. 40, no. 1, pp. 35–42, 2011.
- [25] E. Jones and J. Steinhardt, “Capturing Failures of Large Language Models via Human Cognitive Biases,” *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, pp. 11785–11799, 2022.
- [26] J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu, “RoFormer: Enhanced Transformer with Rotary Position Embedding,” *Neurocomputing*, vol. 568, p. 127063, 2024.
- [27] X. Liu, G. Chen, and W. Wang, “SinkTrack: Attention Sink based Context Anchoring for Large Language Models,” in *International Conference on Learning Representations (ICLR)*, 2026.

- [28] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, and Y. Artzi, “BERTScore: Evaluating Text Generation with BERT,” in *International Conference on Learning Representations (ICLR)*, 2020.
- [29] S. Zhou, U. Alon, S. Agarwal, and G. Neubig, “CodeBERTScore: Evaluating Code Generation with Pretrained Models of Code,” in *Findings of the Association for Computational Linguistics (EMNLP)*, 2023, pp. 13921–13937.
- [30] E. J. Hu *et al.*, “LoRA: Low-Rank Adaptation of Large Language Models,” in *International Conference on Learning Representations (ICLR)*, 2022.
- [31] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, “QLoRA: Efficient Finetuning of Quantized LLMs,” *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, pp. 10088–10115, 2023.
- [32] C. E. Shannon, “A Mathematical Theory of Communication,” *The Bell System Technical Journal*, vol. 27, no. 3, pp. 379–423, 1948.
- [33] T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed. John Wiley & Sons, 2006.